Monads in Practice

Lecture 13

Section 1

Part I: Recall — What Is a Monad?

The Monad type class in Lean

You already use monads in Lean. The full interface comes from three classes:

```
class Pure (m : Type -> Type) where
  pure : a -> m a

class Bind (m : Type -> Type) where
  bind : m a -> (a -> m b) -> m b

class Monad (m : Type -> Type) extends Applicative m, Bind m
```

- `m` is a type constructor (`Option`, `List`, `StateM s`, `IO`)
- `Pure` provides `pure` — lifts a value into the monad
- `Bind` provides `bind` (also written `>>=`) — chains computations
- `Monad` combines them and provides default implementations

An **instance** supplies `pure` and `bind` for a specific type constructor.

The same class in Haskell

In Haskell, the same concept uses slightly different syntax:

```
class Functor m => Monad m where
  return :: a -> m a
  (>>=)  :: m a -> (a -> m b) -> m b
```

Lean	**Haskell**
`pure`	`return`
`bind` / `>>=`	`>>=`
`instance : Monad Option`	`instance Monad Maybe`
do notation	do notation (identical)

We show both syntaxes throughout. The concepts are identical; only the notation differs.

Type classes are not part of Fω

Recall: System Fω gives us the kind system to *state* that `m` has kind $* \to *$. But Fω alone cannot express "and `m` must come equipped with `pure` and `bind`."

A type class is an orthogonal mechanism layered on top. It is a **structure** (a record type) whose fields are functions:

```
structure Monad (m : Type -> Type) where
  pure : a -> m a
  bind : m a -> (a -> m b) -> m b
```

An **instance** is a *term* of this structure — a concrete record inhabiting those fields. In Lean, classes literally *are* structures, and instances *are* terms.

The compiler passes the record implicitly wherever a `[Monad m]` constraint appears. In raw Fω, you would pass `pure` and `bind` as explicit function arguments.

You already use monads in Lean

Every time you write `by` in a proof, you enter the `TacticM` monad:

- `TacticM` carries the proof state (goals, hypotheses, metavariables)
- Each tactic is a function that transforms this state
- `bind` chains tactics: run the first, feed its updated state to the second
- `do` notation lets you write tactic sequences imperatively

Similarly, every `IO` program you write in Lean is monadic. What we are doing today is understanding *how the mechanism works* and how to build your own instances.

The composition problem: why we need bind

In a pure world, composing $f : A \to B$ and $g : B \to C$ is trivial:

$$g \circ f : A \to C \qquad (g \circ f)(x) = g(f(x))$$

The composition problem: why we need bind

In a pure world, composing $f : A \to B$ and $g : B \to C$ is trivial:

$$g \circ f : A \to C \qquad (g \circ f)(x) = g(f(x))$$

Now suppose both functions have effects. Their types become $f : A \to T(B)$ and $g : B \to T(C)$, where T is the monad.

The problem: f produces a $T(B)$, but g expects a plain B. We cannot compose them directly.

`bind` is the glue: it extracts the value from $T(B)$ (handling whatever effect T represents) and feeds it to g.

Bind is the glue

$$\texttt{bind} : T(A) \to (A \to T(B)) \to T(B)$$

Given $f : A \to T(B)$ and $g : B \to T(C)$, we compose them as:

$$\texttt{bind}\ (f\ a)\ g\ :\ T(C)$$

This is *not* the same as unwrapping T and discarding the effect. Bind **combines** the effects:

Monad	**What bind does with the effect**
`Option`	if f fails, skip g entirely
`List`	apply g to every result of f, collect all
`StateM s`	thread the updated state from f into g
`IO`	sequence the side effects of f before those of g

The monad laws

Not every definition of `pure` and `bind` makes a valid monad. The instance must satisfy three laws:

$$\begin{aligned}
\texttt{pure}\ a \gg\!= f &= f\ a && \text{(left unit)} \\
m \gg\!= \texttt{pure} &= m && \text{(right unit)} \\
(m \gg\!= f) \gg\!= g &= m \gg\!= (\lambda x.\ f\ x \gg\!= g) && \text{(associativity)}
\end{aligned}$$

In words: `pure` does nothing when bound (left and right), and chains of binds can be regrouped freely.

Where the laws come from

Recall from last lecture: effectful programs $A \to T(B)$ form a category (the **Kleisli category**) with `pure` as identity and `bind` as composition. The monad laws are exactly the **category laws**:

Monad law	**Category law**
left unit	$\texttt{id} \circ f = f$
right unit	$f \circ \texttt{id} = f$
associativity	$(h \circ g) \circ f = h \circ (g \circ f)$

If the laws fail, composition of effectful programs is not well-behaved: regrouping a chain of operations could change its meaning.

The laws are your responsibility

Important: neither the Lean nor the Haskell compiler checks the monad laws. The type class mechanism only checks that you provide `pure` and `bind` with the right *types*.

In Lean, you *can* state and prove the laws via `LawfulMonad`:

```
class LawfulMonad (m : Type -> Type) [Monad m]
    extends LawfulApplicative m where
  pure_bind  : forall (x : a) (f : a -> m b),
                 pure x >>= f = f x
  bind_assoc : forall (x : m a) (f : a -> m b) (g : b -> m c),
                 x >>= f >>= g = x >>= fun a => f a >>= g
```

But this is a *separate* class you opt into. In Haskell there is no such mechanism — the programmer is simply trusted.

Section 2

Part II: The Option / Maybe Monad

Option: the type

You know this type well. In Lean it is `Option`; in Haskell, `Maybe`.

Lean:

```
inductive Option (a : Type) where
  | none : Option a
  | some : a -> Option a
```

Haskell:

```
data Maybe a = Nothing | Just a
```

As a monad, `Option`/`Maybe` models **computations that can fail**: the effect is "might not produce a result."

The Option instance (Lean)

```
instance : Monad Option where
  pure x := some x
  bind
    | none,   _ => none
    | some x, f => f x
```

- `pure x = some x` — a pure value succeeds trivially
- `bind none _ = none` — first computation failed? Skip the second
- `bind (some x) f = f x` — succeeded? Feed the value to `f`

The Maybe instance (Haskell)

The same instance in Haskell syntax:

```haskell
instance Monad Maybe where
  return x       = Just x
  Nothing >>= f = Nothing
  Just x  >>= f = f x
```

This is the entire implementation. Two lines of pattern matching define how failure propagates through an arbitrarily long chain of computations.

The Lean and Haskell instances are line-for-line the same logic; only the names differ (`some`/`Just`, `none`/`Nothing`).

Option in action

Suppose we have lookup functions that can fail:

Lean:

```
def lookupStudent : String -> Option Student  := ...
def lookupCourse  : Student -> Option Course  := ...
def lookupGrade   : Course -> Option Nat      := ...
```

Haskell:

```
lookupStudent :: String -> Maybe Student
lookupCourse  :: Student -> Maybe Course
lookupGrade   :: Course -> Maybe Int
```

Each can fail. Without monads, you must check each case manually.

Option: chaining with bind

Using >>= to chain them — **Lean:**

```
def getGrade (name : String) : Option Nat :=
  lookupStudent name >>= fun s =>
  lookupCourse s     >>= fun c =>
  lookupGrade c
```

Haskell:

```
getGrade name =
  lookupStudent name >>= \s ->
  lookupCourse s     >>= \c ->
  lookupGrade c
```

If any lookup returns `none`/`Nothing`, the rest is skipped automatically.

Option: tracing the execution

Trace for `name = "Alice"` (enrolled, grade 85):

```
lookupStudent "Alice" ⇒ some alice

some alice >>= lookupCourse ⇒ some cs101

some cs101 >>= lookupGrade ⇒ some 85
```

Now if `lookupStudent "Bob"` returns `none`:

```
none >>= lookupCourse ⇒ none

none >>= lookupGrade ⇒ none
```

Failure propagates automatically. No boilerplate error checking.

Option: do-notation

Both languages have do-notation as syntactic sugar for `bind`:

Lean:

```
def getGrade (name : String) : Option Nat := do
  let student <- lookupStudent name
  let course  <- lookupCourse student
  lookupGrade course
```

Haskell:

```
getGrade name = do
  student <- lookupStudent name
  course  <- lookupCourse student
  lookupGrade course
```

Read `let x <- m` as: "run `m`; if it succeeds, call the result `x`.''

Section 3

Part III: The List Monad

Lists as nondeterministic computations

The type `List a` / `[a]` can model **nondeterminism**: a computation that may return zero, one, or many results.

`Option` / `Maybe`	at most one outcome
`List` / `[]`	any number of outcomes (including zero)

The List instance

Lean 4 provides `Pure` and `Bind` for `List` but not a full `Monad` instance. We provide one:

Lean:

```
instance : Monad List where
  pure x := [x]
  bind xs f := xs.flatMap f
```

Haskell:

```
instance Monad [] where
  return x = [x]
  xs >>= f = concatMap f xs
```

Apply `f` to every element, each producing a list, then flatten. If `xs` = $[x_1, x_2, x_3]$:

$$[x_1, x_2, x_3] \gg\!= f \;=\; f\ x_1 +\!\!+ f\ x_2 +\!\!+ f\ x_3$$

List: a worked example

Compute all pairs by picking one element from each of two lists:

Lean:

```
def pairs : List (Nat * Char) := do
  let x <- [1, 2, 3]
  let y <- ['a', 'b']
  pure (x, y)
```

Haskell:

```
pairs = do
  x <- [1, 2, 3]
  y <- ['a', 'b']
  return (x, y)
```

The do-notation is nearly identical in both languages.

List: tracing the example

For `x = 1`: `['a','b'].flatMap (fun y => [(1,y)])` gives `[(1,'a'), (1,'b')]`

For `x = 2`: `[(2,'a'), (2,'b')]`

For `x = 3`: `[(3,'a'), (3,'b')]`

Concatenated:
`[(1,'a'),(1,'b'),(2,'a'),(2,'b'),(3,'a'),(3,'b')]`

Each `let x <- [1,2,3]` branches into three paths; each `let y <- ['a','b']` branches again. The list monad explores the full Cartesian product.

List: Pythagorean triples (Haskell)

A classic example — find all Pythagorean triples up to n:

```
triples n = do
  a <- [1..n];  b <- [a..n];  c <- [b..n]
  if a*a + b*b == c*c then return (a,b,c) else []
```

`triples 20` gives
`[(3,4,5),(5,12,13),(6,8,10),(8,15,17),(9,12,15)]`

Returning `[]` prunes a branch; `return x` keeps it.

List: Pythagorean triples (Lean)

The same program in Lean:

```
def triples (n : Nat) : List (Nat * Nat * Nat) := do
  let a <- List.range' 1 n
  let b <- List.range' a (n + 1 - a)
  let c <- List.range' b (n + 1 - b)
  if a*a + b*b == c*c then pure (a, b, c) else []
```

`List.range' start count` gives `[start, ..., start+count-1]`.

The logic is line-for-line the same: do-notation, monadic binding with `let x <- xs`, and `[]` to prune branches. Only the range syntax differs.

Comparing Option and List

	Option / Maybe	**List / []**
`pure x`	`some x` / `Just x`	`[x]`
>>= on fail/empty	short-circuit	produce `[]`
>>= on success	apply f to the value	apply f to all, flatten
Effect	partiality	nondeterminism

The do-notation code looks identical in both; only the instance determines the behavior.

Section 4

Part IV: The State Monad

Recall: stateful computations as functions

A computation that reads and writes state of type s, producing a result of type a, is a function $s \to (a, s)$. In Lean this is `StateM s a`; in Haskell, `State s a`.

Lean:

```
def StateT (s : Type) (m : Type -> Type) (a : Type) :=
  s -> m (a * s)

abbrev StateM (s : Type) := StateT s Id
-- so StateM s a = s -> a * s
```

Haskell:

```
newtype State s a = State { runState :: s -> (a, s) }
```

A value of type `StateM Nat String` is a function Nat $\to$ String $\times$ Nat waiting to be called. It has not run yet.

The State instance

Lean (simplified):

```
instance : Monad (StateM s) where
  pure a := fun s => (a, s)
  bind m f := fun s => let (a, s') := m s;  f a s'
```

Haskell:

```
instance Monad (State s) where
  return a = State (\s -> (a, s))
  m >>= f  = State (\s ->
    let (a, s') = runState m s in runState (f a) s')
```

`pure a`: leave state unchanged, produce `a`. ; `bind m f`: run `m` on `s`, get `(a, s')`, then run `f a` on `s'`.

State: the primitive operations

We need ways to **read** and **write** the state. **Lean:**

```
def get : StateM s s :=                fun s => (s, s)
def set (s' : s) : StateM s Unit := fun _ => ((), s')
def modify (f : s -> s) : StateM s Unit := fun s => ((), f s)
```

Haskell:

```
get       = State (\s -> (s, s))
put s'    = State (\_  -> ((), s'))
modify f = State (\s -> ((), f s))
```

State: a counter example

Lean:

```
def incr : StateM Nat Unit := modify fun x => x + 1
def threeIncrements : StateM Nat Unit := do incr; incr; incr
#eval threeIncrements.run 0    -- ((), 3)
```

Haskell:

```
incr = modify (+1)
threeIncrements = do incr; incr; incr
runState threeIncrements 0      -- ((), 3)
```

Note: `threeIncrements` is a pure function. No mutation has occurred; we described a computation and then ran it.

State: Fibonacci

Imperative-style Fibonacci. **Lean:**

```
def fib (n : Nat) : Nat :=
  let loop : StateM (Nat * Nat) Nat := do
    for _ in [:n] do
      let (a, b) <- get;  set (b, a + b)
    let (a, _) <- get;  pure a
  (loop.run (0, 1)).1
```

Haskell:

```
fib n = fst $ runState loop (0, 1) where
  loop = do forM_ [1..n] $ \_ -> do
              (a, b) <- get; put (b, a + b)
            (a, _) <- get; return a
```

Trace for `fib 5`: $(0,1) \to (1,1) \to (1,2) \to (2,3) \to (3,5) \to (5,8)$.
Result: 5 ; (✓).

State: tracing the bind

Let us trace `bind get (fun s => set (s+1))` with initial state 5:

```
bind m f = fun s =>
  let (a, s') := m s          -- get 5 = (5, 5)
  f a s'                    ;; -- set 6 5 = ((), 6)
```

Result: `((), 6)`

The state 5 was read, incremented to 6, and stored — all via pure function composition. No mutation occurred.

Section 5

Part V: The IO Monad

IO: the opaque monad

IO α represents a computation that interacts with the outside world. Both languages have it.

Lean:

```
IO.println  : String -> IO Unit
IO.FS.readFile : System.FilePath -> IO String
```

Haskell:

```
putStrLn :: String -> IO ()
readFile :: FilePath -> IO String
```

Unlike `Option` and `List`, you **cannot pattern match** on an `IO` value. `IO` is abstract by design. The only way to combine IO actions is through `pure` and `bind`.

IO: a simple program

Lean:

```
def main : IO Unit := do
  IO.println "What is your name?"
  let stdin <- IO.getStdin
  let name <- stdin.getLine
  IO.println s!"Hello, {name}!"
```

Haskell:

```
main :: IO ()
main = do
  putStrLn "What is your name?"
  name <- getLine
  putStrLn ("Hello, " ++ name ++ "!")
```

Each line in the do-block desugars to a `bind`. The syntax is nearly identical.

Pure monads vs. IO

`Option`, `List`, and `StateM` are **pure**: they simulate effects by composing ordinary functions. You can run them yourself.

`IO` is different. The state is the entire real world. Only the runtime executes IO actions, by running `main`.

	Pure monads	**IO**
State	a value you provide	the real world
Run it yourself?	yes (`.run`, etc.)	no (only `main`)
Deterministic?	same input $\Rightarrow$ same output	no

Section 6

Part VI: Putting It All Together

The pattern

Every monad instance follows the same structure:

1. Pick a type constructor $T : \text{Type} \to \text{Type}$
2. Define `pure : a -> T a` (embed a pure value)
3. Define `bind : T a -> (a -> T b) -> T b` (compose)
4. Verify the three monad laws (optionally prove via `LawfulMonad`)

Once you have an instance, **all generic monad machinery works**: do-notation, `List.mapM`, `forM`, `sequence`, ...

You write these combinators *once*, and they work for every monad.

Summary of instances

Lean	**Haskell**	`pure a`	`bind`	**Effect**
`Option`	`Maybe`	`some a`	skip/apply	failure
`List`	`[]`	`[a]`	flatMap	nondet.
`StateM s`	`State s`	λs.(a,s)	thread state	mut. state
`IO`	`IO`	yield a	sequence	real world

Four computational patterns, one uniform interface. The do-block structure works regardless of which monad you are in.

Generic code: an example

```
def mapM' [Monad m] (f : a -> m b) : List a -> m Unit
  | []      => pure ()
  | x :: xs => f x >>= fun _ => mapM' f xs
```

This works for *any* monad. Instantiate:

- `m = IO`: performs an IO action on each element
- `m = Option`: applies a failable operation; short-circuits on failure
- `m = StateM s`: applies a stateful operation, threading state
- `m = List`: nondeterministic operation, collecting all outcomes

You wrote `mapM'` once. The instance determines the behavior.

Section 7

Summary

What we covered

1. **Recall**: the `Monad` class in Lean (and Haskell); monads in Lean's tactic mode
2. **Bind as glue**: composing effectful programs $A \to T(B)$ and $B \to T(C)$
3. **Option / Maybe**: the instance, failure propagation, do-notation
4. **List**: the instance, nondeterminism, Pythagorean triples
5. **StateM / State**: the instance, get/set, Fibonacci
6. **IO**: the opaque monad, sequencing, pure vs. IO

Next time

Next lecture: type inference.

System F's type inference is undecidable (Wells, 1999). The fix is the **Hindley–Milner** type system: restrict to prenex polymorphism, allow generalization only at `let`-bindings, and use **unification** as the engine.

This is what makes all of the Haskell code we wrote today work without explicit type annotations. Lean uses a more powerful elaboration algorithm, but the core idea — unification — is the same.